

Django CRUD Project

June 8, 2026

1 Django CRUD Project

1.1 Setup Django Project

1. Make sure Django is installed:

```
pip install django
```

1.2 Create a new Django project:

```
django-admin startproject student_crud  
cd student_crud
```

1.3 Create a new app:

```
python manage.py startapp students
```

1.4 Project Structure Overview

```
student_crud/
```

```
    student_crud/  
        settings.py  
        urls.py  
        ...
```

```
    students/  
        models.py  
        views.py  
        forms.py  
        urls.py  
        templates/  
            students/  
                student_list.html  
                student_form.html  
                student_confirm_delete.html
```

```
manage.py
```

1.5 Project Settings

In student_crud/settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'students', # Add our app  
]
```

1.6 Create the Model

In students/models.py:

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    roll = models.IntegerField()  
    city = models.CharField(max_length=100)  
  
    def __str__(self):  
        return f"{self.name} ({self.roll})"
```

1.7 Create Database tables

- Run migrations:

```
python manage.py makemigrations  
python manage.py migrate
```

1.8 Create a ModelForm

In students/forms.py:

```
from django import forms  
from .models import Student  
  
class StudentForm(forms.ModelForm):  
    class Meta:  
        model = Student  
        fields = ['name', 'roll', 'city']
```

1.9 Create Function-Based Views (CRUD)

In students/views.py:

```

from django.shortcuts import render, redirect, get_object_or_404
from .models import Student
from .forms import StudentForm

# List all students
def student_list(request):
    students = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students})

# Add new student
def student_create(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = StudentForm()
    return render(request, 'students/student_form.html', {'form': form})

# Update student
def student_update(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        form = StudentForm(request.POST, instance=student)
        if form.is_valid():
            form.save()
            return redirect('student_list')
    else:
        form = StudentForm(instance=student)
    return render(request, 'students/student_form.html', {'form': form})

# Delete student
def student_delete(request, pk):
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        student.delete()
        return redirect('student_list')
    return render(request, 'students/student_confirm_delete.html', {'student': student})

```

1.10 Step 8: Configure URLs

1.10.1 student_crud/urls.py:

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [

```

```

    path('admin/', admin.site.urls),
    path('', include('students.urls')), # Include app URLs
]

```

1.11 students/urls.py:

```

from django.urls import path
from . import views

```

```

urlpatterns = [
    path('', views.student_list, name='student_list'),
    path('create/', views.student_create, name='student_create'),
    path('update/<int:pk>/', views.student_update, name='student_update'),
    path('delete/<int:pk>/', views.student_delete, name='student_delete'),
]

```

1.12 Create Templates

- Create a folder structure: students/templates/students/

1.12.1 student_list.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Student List</title>
</head>
<body>
    <h1>Students</h1>
    <a href="{% url 'student_create' %}">Add New Student</a>
    <table border="1">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Roll</th>
            <th>City</th>
            <th>Actions</th>
        </tr>
        {% for student in students %}
        <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.roll }}</td>
            <td>{{ student.city }}</td>
            <td>
                <a href="{% url 'student_update' student.id %}">Edit</a> |
                <a href="{% url 'student_delete' student.id %}">Delete</a>
            </td>
        </tr>
    </table>

```

```

        {% endfor %}
    </table>
</body>
</html>

```

1.13 student_form.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Student Form</title>
</head>
<body>
    <h1>Student Form</h1>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Save</button>
    </form>
    <a href="{% url 'student_list' %}">Back to List</a>
</body>
</html>

```

1.14 student_confirm_delete.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Delete Student</title>
</head>
<body>
    <h1>Are you sure you want to delete "{{ student.name }}"?</h1>
    <form method="POST">
        {% csrf_token %}
        <button type="submit">Yes, Delete</button>
    </form>
    <a href="{% url 'student_list' %}">Cancel</a>
</body>
</html>

```

1.15 Run the Server

python manage.py runserver

Visit <http://127.0.0.1:8000/> to see your CRUD app in action:

- Add new students
- Update existing students
- Delete students
- View all students